Authentication API Reference

Complete reference for the authentication module — endpoints, payloads, token lifecycle, roles, and error handling.

Table of Contents

Base URL

```
http://localhost:4000/api/v1
```

All endpoints below are relative to this base.

Response Envelope

Every response follows a consistent JSON envelope:

Success Response

```
{
  "success": true,
  "message": "Human-readable message",
  "data": { ... }
}
```

Success with Pagination

```
{
  "success": true,
  "message": "...",
  "data": [ ... ],
  "meta": {
    "page": 1,
    "limit": 20,
    "total": 45,
    "pages": 3
  }
}
```

Error Response

```
{
  "success": false,
  "message": "Error description"
}
```

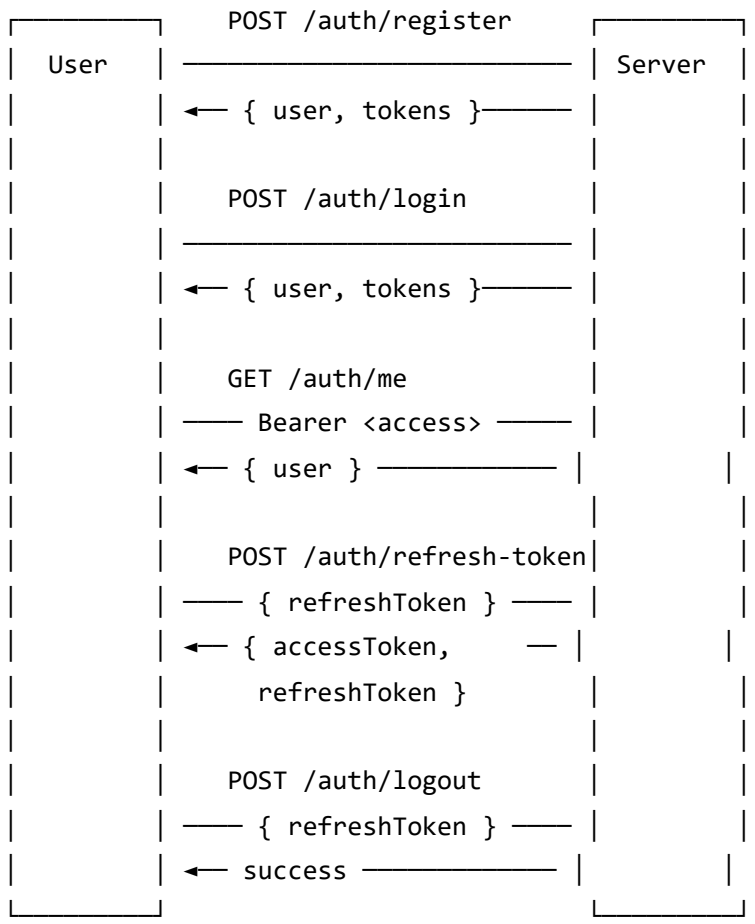
Validation Error Response

```
{
  "success": false,
  "message": "Validation error",
  "errors": [
    { "field": "email", "message": "Invalid email address" },
    { "field": "password", "message": "Password must be at least 8 characters" }
  ]
}
```

Error Handling

HTTP Status	Meaning	When It Happens
`400`	Bad Request	Invalid input format, malformed ID
`401`	Unauthorized	Missing/expired/invalid token, wrong credentials
`403`	Forbidden	Account suspended/blocked, insufficient role
`404`	Not Found	Resource doesn't exist
`409`	Conflict	Duplicate email, wallet already linked
`422`	Unprocessable Entity	Validation failed (check `errors` array)
`429`	Too Many Requests	Rate limit exceeded
`500`	Internal Server Error	Server bug (report to backend team)

Authentication Flow Overview



Key concepts:

1. **Register** or **Login** returns both `accessToken` and `refreshToken`
2. Use `accessToken` in the `Authorization` header for all protected requests
3. When `accessToken` expires, call **Refresh Token** with the `refreshToken`
4. The refresh endpoint **rotates** both tokens (old refresh token is invalidated)
5. On **Logout**, send the `refreshToken` to invalidate the session server-side

Endpoints

1. Register

Creates a new account and returns an authenticated session.

```
POST /auth/register
```

Rate limited: Yes

Request Body:

```
{
  "name": "John Doe",
  "email": "john@example.com",
  "password": "MyPassword1",
  "role": "tenant"
}
```

Field	Type	Required	Notes
`name`	string	✅	2–100 characters
`email`	string	✅	Valid email, must be unique
`password`	string	✅	Min 8 chars, at least 1 uppercase, 1 number
`role`	string	❌	`"property_owner"` or `"tenant"` (default: `"tenant"`)

> **Note**: `admin` and `super_admin` roles cannot be self-registered. They are created by the super admin.

Success Response (201):

```
{
  "success": true,
  "message": "Account created successfully",
  "data": {
    "user": {
      "id": "6670a1b2c3d4e5f6a7b8c9d0",
      "name": "John Doe",
      "email": "john@example.com",
      "role": "tenant",
      "phone": null,
      "profileImage": null,
      "accountStatus": "pending",
      "kycStatus": "not_started",
      "emailVerified": false,
      "mustResetPassword": false,
      "walletAddress": null,
      "walletStatus": "unlinked"
    },
    "tokens": {
      "accessToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
      "refreshToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
    }
  }
}
```


Error Cases:

Status	Message	Cause
409	Email already registered	Duplicate email
422	Validation error	Invalid input (see `errors` array)

2. Login

Authenticates a user and returns a session.

```
POST /auth/login
```

Rate limited: Yes

Request Body:

```
{
  "email": "john@example.com",
  "password": "MyPassword1"
}
```

Field	Type	Required
email	string	✅
password	string	✅

Success Response (200):

```
{
  "success": true,
  "message": "Login successful",
  "data": {
    "user": {
      "id": "6670a1b2c3d4e5f6a7b8c9d0",
      "name": "John Doe",
      "email": "john@example.com",
      "role": "tenant",
      "accountStatus": "active",
      "kycStatus": "not_started",
      "emailVerified": false,
      "mustResetPassword": false,
      "walletAddress": null,
      "walletStatus": "unlinked"
    },
    "tokens": {
      "accessToken": "eyJhbG...",
      "refreshToken": "eyJhbG..."
    }
  }
}
```

Error Cases:

Status	Message	Cause
401	Invalid email or password	Wrong credentials
403	Account is suspended	Account status prevents login
403	Account is blocked	Permanently blocked

> **Important**: After login, check `user.mustResetPassword`. If `true`, redirect the user to the change password screen before allowing access to the app.

3. Get Current User

Returns the authenticated user's profile.

```
GET /auth/me
```

Headers:

```
Authorization: Bearer <accessToken>
```

Success Response (200):

```
{
  "success": true,
  "message": "Profile fetched",
  "data": {
    "id": "6670a1b2c3d4e5f6a7b8c9d0",
    "name": "John Doe",
    "email": "john@example.com",
    "role": "tenant",
    "phone": "+251900000001",
    "profileImage": null,
    "accountStatus": "active",
    "kycStatus": "not_started",
    "emailVerified": false,
    "mustResetPassword": false,
    "walletAddress": null,
    "walletStatus": "unlinked"
  }
}
```


4. Refresh Token

Rotates the token pair. The old refresh token is **invalidated** after this call.

```
POST /auth/refresh-token
```

Rate limited: Yes

Request Body:


```
{
  "refreshToken": "eyJhbG..."
}
```


Success Response (200):

```
{
  "success": true,
  "message": "Tokens refreshed",
  "data": {
    "accessToken": "eyJhbG...(new)",
    "refreshToken": "eyJhbG...(new)"
  }
}
```

Error Cases:

Status	Message	Cause
401	Invalid or expired refresh token	Token verification failed
401	Session not found	Token was already used or doesn't exist
401	Refresh token reuse detected	Old token replayed — **all sessions in the family are revoked**
401	Session expired	Refresh token TTL exceeded

> **Security**: If a refresh token is reused (replay attack), the server revokes **all** sessions in that token family. The user must log in again.

5. Logout

Invalidates a single session.

```
POST /auth/logout
```

Request Body:

```
{
  "refreshToken": "eyJhbG..."
}
```

Success Response (200):

```
{
  "success": true,
  "message": "Logged out",
  "data": null
}
```

> This endpoint does **not** require the `Authorization` header — it only needs the refresh token. Always call this when the user logs out to clean up the server session.

6. Logout All Sessions

Revokes all active sessions for the authenticated user.

```
POST /auth/logout-all
```

Headers:

```
Authorization: Bearer <accessToken>
```

Success Response (200):

```
{
  "success": true,
  "message": "Logged out of all sessions",
  "data": null
}
```

> Use this for "sign out everywhere" functionality.

7. List Sessions

Returns a list of the user's active sessions with device/IP metadata.

```
GET /auth/sessions
```

Headers:

```
Authorization: Bearer <accessToken>
```

Success Response (200):

```
{
  "success": true,
  "message": "Active sessions",
  "data": [
    {
      "id": "...",
      "userAgent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64)...",
      "ip": "192.168.1.10",
      "createdAt": "2026-06-16T12:00:00.000Z",
      "expiresAt": "2026-07-16T12:00:00.000Z"
    }
  ]
}
```

8. Update Profile

Updates the user's name, phone, or profile image.

```
PATCH /auth/profile
```

Headers:

```
Authorization: Bearer <accessToken>
```

Request Body (all fields optional, at least one required):

```
{
  "name": "John Updated",
  "phone": "+251911000000",
  "profileImage": "https://res.cloudinary.com/..."
}
```

Field	Type	Notes
`name`	string	2–100 characters
`phone`	string	Max 20 characters
`profileImage`	string	Valid URL

Success Response (200):

```
{
  "success": true,
  "message": "Profile updated",
  "data": { ... }
}
```

9. Change Password

Changes the authenticated user's password. Requires the current password.

```
POST /auth/change-password
```

Headers:

```
Authorization: Bearer <accessToken>
```

Request Body:

```
{
  "currentPassword": "MyOldPassword1",
  "newPassword": "MyNewPassword2"
}
```

Field	Type	Required	Notes
`currentPassword`	string	✅	Must match current password
`newPassword`	string	✅	Min 8 chars, 1 uppercase, 1 number

Success Response (200):

```
{
  "success": true,
  "message": "Password changed; please sign in again",
  "data": null
}
```

> **Important**: After a password change, all existing sessions are invalidated. The frontend should redirect to login.

10. Forgot Password

Initiates a password reset flow by sending a reset link/token to the user's email.

```
POST /auth/forgot-password
```

Rate limited: Yes (stricter limit)

Request Body:

```
{
  "email": "john@example.com"
}
```

Success Response (200):

```
{
  "success": true,
  "message": "If an account with that email exists, a reset link has been sent",
  "data": null
}
```

> **Security**: The response is always `200` regardless of whether the email exists to prevent email enumeration attacks.

11. Reset Password

Completes the password reset using the token received via email.

```
POST /auth/reset-password
```

Rate limited: Yes (stricter limit)

Request Body:

```
{
  "token": "abc123resettoken...",
  "newPassword": "MyNewPassword2"
}
```

Field	Type	Required	Notes
`token`	string	✅	Token from the reset email
`newPassword`	string	✅	Min 8 chars, 1 uppercase, 1 number

Success Response (200):

```
{
  "success": true,
  "message": "Password reset successful; please sign in again",
  "data": null
}
```

12. Wallet: Request Challenge

Requests a nonce message that the user must sign with their Ethereum wallet to prove ownership.

```
POST /auth/wallet/challenge
```

Headers:

```
Authorization: Bearer <accessToken>
```

Request Body:

```
{
  "walletAddress": "0x742d35Cc6634C0532925a3b844Bc9e7595f2bD18"
}
```

Success Response (200):


```
{
  "success": true,
  "message": "Wallet challenge created",
  "data": {
    "walletAddress": "0x742d35cc6634c0532925a3b844bc9e7595f2bd18",
    "message": "Real Estate Marketplace\n\nLink wallet 0x742d...f2bd18 to account john@example.
    "expiresAt": "2026-06-16T12:10:00.000Z"
  }
}
```


> The returned `message` is what the user must sign in MetaMask or their wallet app. The challenge expires in 10 minutes.

13. Wallet: Link

Submits the signed challenge to link the wallet to the user's account.

```
POST /auth/wallet/link
```

Headers:

```
Authorization: Bearer <accessToken>
```

Request Body:

```
{
  "walletAddress": "0x742d35Cc6634C0532925a3b844Bc9e7595f2bD18",
  "signature": "0x1234...abcd"
}
```

Success Response (200):

```
{
  "success": true,
  "message": "Wallet linked",
  "data": {
    "id": "...",
    "walletAddress": "0x742d35cc6634c0532925a3b844bc9e7595f2bd18",
    "walletStatus": "linked",
    ...
  }
}
```

Error Cases:

Status	Message	Cause
400	Invalid wallet address	Malformed EVM address
409	Wallet already linked to another account	Address in use

14. Wallet: Unlink

Removes the linked wallet from the user's account.

```
DELETE /auth/wallet
```

Headers:

```
Authorization: Bearer <accessToken>
```

Success Response (200):

```
{
  "success": true,
  "message": "Wallet unlinked",
  "data": {
    "id": "...",
    "walletAddress": null,
    "walletStatus": "unlinked",
    ...
  }
}
```

Token Management

JWT Structure

The access token payload contains:

```
{
  "userId": "6670a1b2c3d4e5f6a7b8c9d0",
  "email": "john@example.com",
  "role": "tenant",
  "iat": 1718540000,
  "exp": 1718543600
}
```

Token Refresh Behavior

- Present the `refreshToken` to `POST /auth/refresh-token`
- The server **rotates** the pair: old refresh token is revoked, new pair is issued
- If a revoked refresh token is replayed (reuse detection), the **entire session family** is revoked
- The consumer must store and use the **new** tokens from the refresh response

Role-Based Access

Role	Value	Can Self-Register	Description
Super Admin	`super_admin`	❌	Full system access, manages admins
Admin	`admin`	❌	Reviews listings, KYC, manages users
Property Owner	`property_owner`	✅	Creates/manages listings, responds to offers
Tenant	`tenant`	✅	Browses, submits offers, rental applications

Account States

The `accountStatus` field determines what the user can do:

Status	Can Login	Description
`pending`	✅	Newly registered, awaiting verification
`active`	✅	Fully active account
`suspended`	❌	Temporarily disabled by admin
`blocked`	❌	Permanently disabled
`rejected`	❌	Account verification was rejected

`mustResetPassword` flag: When `true`, the user was assigned a temporary password by an admin. The consumer should enforce a password change before granting full access.

Password Policy

Rule	Requirement
Minimum length	8 characters
Uppercase letter	At least 1
Number	At least 1

These rules are enforced server-side via Joi validation.

Rate Limiting

Endpoint Group	Window	Max Requests
Auth endpoints (login, register, refresh)	Configured per env	Default limit
Password reset (forgot, reset)	Stricter	Lower limit
All other endpoints	Standard	Standard limit

When rate limited, the server returns:

```
{
  "success": false,
  "message": "Too many requests, please try again later"
}
```

HTTP Status: `429`

Test Credentials

Role	Email	Password
Super Admin	superadmin@realestate.dev	SuperAdmin1!

Role	Email	Password
Admin	admin@realestate.dev	PlatformAdmin1!
Property Owner	owner@realestate.dev	PropertyOwner1!
Property Owner	abebe@realestate.dev	AbebeOwner1!
Property Owner	tigist@realestate.dev	TigistOwner1!
Property Owner	dawit@realestate.dev	DawitOwner1!
Property Owner	sara@realestate.dev	SaraOwner1!
Tenant	tenant@realestate.dev	TenantUser1!

> These are seeded via `npm run seed:users` in the backend project.

Swagger / OpenAPI

Interactive API documentation is available at:

```
http://localhost:4000/api/v1/docs/
```

Raw OpenAPI JSON spec:

```
http://localhost:4000/api/v1/docs.json
```